Browser Automation Project: Study Guide

Bravebird Automation 2 · implementation walkthrough

15 October 2026

- Browser Automation Project: Study Guide
  - 1. What the project does
  - 2. How to trace a run
  - 3. Repository map
  - 4. Configuration: portal versus business data
  - 5. CLI and command flow
  - 6. Session validation and recovery
  - 7. How the second factor works
  - 8. Locator resolution
  - 9. Portal workflow and handled quirks
  - 10. Retry and failure policy
  - 11. Document capture and manifest
  - 12. The local API and browser UI
  - 13. File-by-file code guide
  - 14. Why this approach, and what changes at one hundred portals?
  - 15. Submission and review checklist

Browser Automation Project: Study Guide

Bravebird Automation 2 · implementation walkthrough · 15 October 2026

This guide explains the checked-in implementation file by file. It distinguishes the real GitHub workflow from the disposable local fixture so that a reviewer can tell which claims were tested against a real account and which were tested locally.

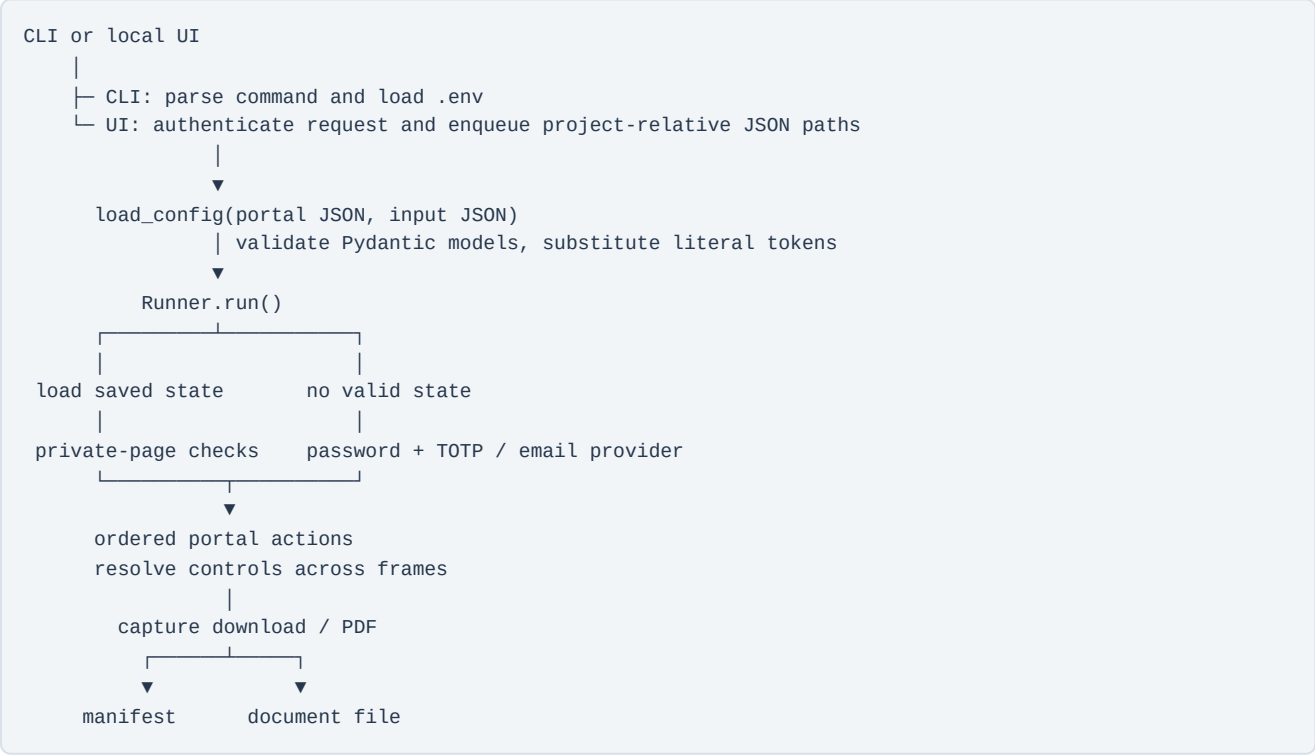
1. What the project does

The runner signs into GitHub with the account's native password and authenticator-app TOTP, validates and reuses browser state, visits the selected repository's Releases page, fills the new-release form, and prints the preview page to PDF. It leaves the form unpublished. The current input points to `ckiran24/shopping_bot`.

The run produces a JSON manifest with the redacted inputs, session decision, visited pages, stage timings, locator attempts, and captured-file metadata. If a browser-stage step fails, it writes a failure manifest and a sanitized evidence bundle. A local portal demonstrates the same mechanics without contacting GitHub or using an account.

The live acceptance matrix passed on the user's Windows machine: a fresh login, a reused session, recovery from corrupted saved state, and a changed-input run. The user confirmed the generated outputs worked. The real run artifacts remain on that machine. Locally, the mutated-HTML resolver, sanitized failure evidence, fixture portal, queue API, and browser UI have automated coverage. At the last full run, all 17 tests passed. See `VALIDATION.md` for the evidence record.

2. How to trace a run



The CLI and queue both call the same `Runner`. The queue has a single worker, so it serializes work within the API process. It is not a durable distributed queue.

3. Repository map

File or folder	Responsibility
`automation/__main__.py`	CLI parser, `.env` loading, dispatch, safe top-level diagnostics
`automation/config.py`	Reads JSON, performs named substitutions, builds a redactor, validates a portal
`automation/models.py`	Strict Pydantic schemas and cross-field validation for portals, actions, sessions, and targets
`automation/runner.py`	Browser lifecycle, session validation, login, workflow actions, document capture, manifests, retries, failure evidence
`automation/resolver.py`	Shared semantic-to-CSS locator resolver across the main frame and child frames
`automation/otp.py`	TOTP and optional TLS IMAP code providers behind one interface
`automation/security.py`	Redaction, atomic private JSON writes, fail-closed evidence HTML sanitization
`automation/errors.py`	Separates final failures from failures that can safely retry
`automation/browser.py`	Chooses environment-specified, system, or Playwright-managed Chromium
`automation/offline.py`	Runs the production resolver on saved HTML with network requests blocked
`automation/demo.py`	Disposable HTTP portal and nine fixture acceptance scenarios
`automation/api.py`	Loopback HTTP server, bounded sequential queue, token handling, static UI serving
`automation/ui.html`, `ui.js`, `ui.css`	Small local job form, submission/status polling, and presentation
`portals/github.json`	Real GitHub login/session/form locators and ordered actions
`portals/github-email.json`	Optional GitHub password plus email-device-code configuration
`examples/inputs.github*.json`	Business values kept separate from UI selectors and code
`examples/saved-form.html`, `examples/targets.json`	Non-sensitive offline locator fixture and targets
`scripts/setup.sh`	Unix virtual-environment, dependency, and browser setup
`scripts/mutate_html.py`	Mutates a saved form to simulate common DOM changes
`scripts/test-live.ps1`	Runs the four live GitHub acceptance cases without publishing
`tests/test_acceptance.py`	Fixture login, session, TOTP, resolver, retry, capture, and evidence tests
`tests/test_api.py`	Queue/auth/path/UI tests, including a browser-driven fixture job
`requirements.txt`, `requirements.lock`, `pyproject.toml`	Runtime dependency declarations, pinned install lock, and package metadata
`.env.example`, `.gitignore`	Safe environment-variable template and exclusions for secrets/run state

`examples/output/` contains generated output from the synthetic local fixture. `runs/`, `.state/`, `.env`, and `.venv/` are local-only and ignored by Git.

4. Configuration: portal versus business data

The runner accepts two JSON inputs with different purposes:

- `portals/github.json` describes how GitHub behaves: origin, private session page, authentication markers, login controls, locator alternatives, steps, and capture action.
- `examples/inputs.github.json` holds this run's values: owner, repository, tag, branch, title, notes, and filename.

`config.load_config()` reads both files. It substitutes only strings that exactly contain tokens such as `{{inputs.repository}}` or `{{env.PORTAL_USERNAME}}`. The value is inserted as data; it is never evaluated as Python. Unknown or missing variables fail setup. `Redactor` is populated from credentials, TOTP seed, and configured sensitive inputs before the runner writes diagnostics.

`models.py` then validates the rendered portal. Extra JSON properties are rejected. A `Target` needs at least two independent locator descriptions. A `goto` must be a path on the configured origin, a capture needs a safe filename, retries have bounded values, and real non-local origins must use HTTPS. This catches a bad adapter before opening a browser.

Why keep the files separate? A different release title or tag should not require editing selectors. A GitHub redesign should not require changing the business data. Separate files also make it straightforward to test one portal adapter with several inputs.

5. CLI and command flow

`automation/__main__.py` defines five commands:

- `run` loads a portal and input file, constructs `Runner`, and returns a status plus manifest path.
- `check-config` reports missing setting names and validates a TOTP seed without printing secret values.
- `offline` calls `check_html()` against a saved page and target list.
- `demo` runs the fixture portal acceptance scenarios.
- `api` starts the loopback job service.

The entry point loads the project-root `.env` without overriding values already supplied by the process. Top-level errors use a generic safe diagnostic because parser and browser exceptions can include sensitive values. Exit codes distinguish success, runtime failure, configuration failure, and interruption.

6. Session validation and recovery

The state file is a versioned envelope with an origin and Playwright storage state. It is not trusted just because it parses. Startup proceeds as follows:

1. Read JSON, check the version and origin, and validate the cookie/origin structure. Malformed or wrong-origin state is marked corrupt.
2. Create a Chromium context, optionally loading the state.
3. Open the configured private page in a separate tab. Resolve at least two positive markers and, for GitHub, check that the personal-profile link matches the configured username.
4. If configured, check an authenticated endpoint and its expected JSON. HTTP 401/403 means unauthenticated; transient 429/5xx means retry rather than treating the user as logged out.

5. A private page that returns HTTP 200 but contains a session-expired page fails because the expected private content and identity are missing.
6. On invalid saved state, close that context and create a clean one so stale cookies and local storage cannot interfere. Perform a full login.
7. Save state only after positive authentication; refresh it again after the workflow in case cookies rotated.

The separate probe tab matters during login: a failed check does not navigate away from a one-time-code challenge. The state file is sensitive. `private_json()` writes to a restrictive temporary file and atomically replaces the destination; `.state/` is Git-ignored.

7. How the second factor works

Both providers implement the `CodeProvider.get_code(Challenge)` interface. The `Challenge` carries a stage, attempt number, previous rejected code, and request time.

TOTP

`TOTPProvider` reads the Base32 authenticator enrollment seed from `PORTAL_TOTP_SECRET`, registers it with the redactor, and uses PyOTP/RFC 6238 to compute the current code locally. It waits when the code is too close to its 30-second expiry. It does not poll email and does not need the six-digit value pasted into `.env`.

Email over IMAP

The optional `IMAPProvider` connects over TLS to a configured mailbox, selects it read-only, and searches a bounded set of recent messages. It requires an exact sender, expected subject text, a date close to the challenge time, and exactly one distinct code matching the configured regular expression. It prefers plain text and uses `BODY.PEEK[]`, so messages are not marked read. Consumed UIDs and the previous rejected code are excluded. A dropped connection is polled again until the bounded deadline; bad credentials and TLS verification errors fail closed.

Rejection policy

The runner submits a code once and checks for positive authenticated-page evidence. Only a clearly visible rejected-code marker permits one retry. For TOTP, the provider supplies a fresh code from a later time window. If email configuration has a resend action, it requests a fresh message first. A second rejection, absent challenge, unknown SSO origin, or ambiguous network result fails; the runner does not blindly replay a one-time code.

8. Locator resolution

Each target contains multiple independent descriptions. The resolver evaluates one strategy across all current frames before trying the next strategy. That prevents a weak main-page CSS match from beating an accessible-name match inside an iframe. It waits within a configured deadline while the DOM and frame list settle.

The order is:

1. Exact accessible role and name — closest to a user's understanding; usually stable when CSS changes.
2. Associated label — HTML form meaning is more durable than layout or class names.
3. Exact text, then nearby text in a field block — useful when controls have incomplete accessible names.
4. Portal-provided structural relationship — a targeted fallback for unusual markup.

5. CSS alternatives — useful as a last resort, but more fragile during redesigns.

Only one visible match is accepted. Multiple matches are a final ambiguity error, rather than a silent choice of the first control. Every attempt is recorded with target, strategy, frame, and outcome. `LocatorUnavailable` is retryable; an invalid locator definition or ambiguous target is not.

`offline.check_html()` launches the same resolver with JavaScript disabled and all network requests aborted. The test mutation renames classes, reorders siblings, adds a wrapper, and embeds the saved form in `iframe[srcdoc]`. Using the same resolver offline makes the change reproducible and prevents test pages from sending data anywhere.

9. Portal workflow and handled quirks

The GitHub adapter uses this route:

1. Validate or establish the session at the private profile page.
2. Navigate to the configured repository.
3. Open Releases and the new-release page.
4. Open the current “Tag: Select tag” dropdown, choose “Create new tag,” fill the `Tag *` dialog, and select the target branch.
5. Fill release title and notes, wait for dependent/preview content, and print the preview to PDF.

The adapter handles GitHub's current create-tag dialog labels, dependent tag/branch form, redirect or automatic submission after TOTP entry, nested frames in the fixture, and delayed controls. The local fixture additionally exercises native download, a new tab with HTML print content, current-page PDF printing, and one transient 503 response. The real workflow stops before publishing or saving a draft.

10. Retry and failure policy

`errors.py` separates `RetryableFailure` from `FinalFailure`. `Runner.stage_run()` records attempts and duration, and uses bounded exponential backoff. Retries can occur for Playwright timeouts, recognized connection resets/timeouts, HTTP 429/5xx, a temporarily missing locator, enabled-state waits, and dropped IMAP connections. The maximum attempt count is configured per portal and capped by the model.

Fill/select/wait operations are idempotent and may retry. Navigation opts in where safe. Clicks, downloads, printing, and TOTP submissions are not retried implicitly because repeating them may duplicate an external action. A portal action can explicitly mark safe navigation as retryable. Authentication rejection, unsupported prompts, invalid configuration, ambiguous locators, most HTTP 4xx responses, and invalid documents fail without retry.

On a final browser failure, `Runner.evidence()` writes stage/type metadata, resolver history, and an origin-only URL with its path masked. It sanitizes HTML by masking every text node and removing all attributes except a safe input type. It takes the screenshot from a JavaScript-disabled, network-blocked copy. This sacrifices useful text and visual detail to protect unknown account data, not only known passwords.

11. Document capture and manifest

The generic runner supports three capture paths:

- `download`: click inside Playwright's download event and save the file.

- `popup_pdf`: open a popup containing HTML and print that page to PDF.
- `print_pdf`: print the current page to PDF.

Filenames must be safe basenames and cannot overwrite earlier output. Captures must be non-empty; PDFs must start with `%PDF-`. Files are written under `runs/<run>/documents/` and their byte count and SHA-256 are recorded. PDF hashes can differ between otherwise equivalent runs because browser-produced PDFs may contain changing metadata; visible content is the appropriate comparison.

The manifest includes schema/run identifiers, start time, portal name, redacted input object, session state/reuse/full-login flags, visited page origins/paths after redaction, stage attempts and timings, locator attempts, captured-file metadata, status, and failure reference. Secret values are never intentionally included in the manifest or evidence. Live capture documents can contain account-related information and must be reviewed before sharing.

12. The local API and browser UI

`api.py` creates or loads a random bearer token in `.state/api-token.json`. The HTTP server binds to `127.0.0.1`, limits request size, accepts only portal/input JSON files beneath the project root, and requires authorization for job submission/status. `LocalJobService` holds a bounded FIFO and a single worker thread. Jobs and queue contents are memory-only; stopping the process loses unfinished work. Each portal name gets a distinct API session-state file.

The bundled `ui.html`, `ui.js`, and `ui.css` provide a small browser form. The user enters the token, submits project-relative portal/input paths, and the UI polls status until success/failure. It does not store the token in localStorage. The API serves the three static assets with a restrictive content security policy. The browser UI is intentionally a local development interface, not a public multi-user control plane.

The API tests cover unauthenticated rejection, traversal rejection, bounded-queue behavior, health/token-file behavior, an end-to-end fixture run, and a Chromium UI submission/status flow. This is the extra-credit UI/queue work. Proxy routing and concurrent multi-portal execution are not implemented.

13. File-by-file code guide

`automation/__main__.py`

`parser()` defines `run`, `check-config`, `offline`, `demo`, and `api`. `main()` dispatches the selected command, loads the relevant configuration, builds the runner, and prints a machine-readable result. `entrypoint()` loads `.env`, runs the async command, catches interruption, and returns a safe top-level diagnostic instead of raw exception text. This keeps CLI concerns out of the runner and protects secrets in error output.

`automation/config.py`

`load_config(path, inputs_path)` reads the two JSON documents, adds configured sensitive input values to `Redactor`, recursively substitutes only `{{inputs.name}}` and `{{env.NAME}}`, registers environment secrets, then calls `Portal.model_validate`. The literal token grammar is deliberately small: it avoids arbitrary evaluation and makes missing values explicit.

`automation/models.py`

The shared base forbids unknown fields. `Target` describes alternative locator hooks and requires at least two. `Action` restricts supported operations and validates action-specific fields and same-origin paths. `Session`, `Login`, and `Identity` describe authentication/session checks; `Portal` bounds timeouts/retries, validates origin safety, and checks unique action keys. This moves schema errors before browser side effects.

`automation/errors.py`

`FinalFailure` marks conditions that must stop. `RetryableFailure` marks temporary availability problems. `LocatorUnavailable` is a retryable subtype; `AuthenticationFailure` is a final subtype. The type distinction lets `stage_run()` retry only cases intended to be safe.

`automation/browser.py`

`launch_options()` chooses Chromium from `BROWSER_EXECUTABLE`, then system `chromium`, then the Playwright-managed installation. It also chooses headless/headed mode. Centralizing this avoids different launch behavior across the runner and offline tests.

`automation/otp.py`

`Challenge` and `CodeProvider` define the common provider contract. `TOTPProvider` validates the seed and generates a code at a safe point in its time window. `IMAPProvider` validates settings, parses messages, filters sender/subject/date/code, polls with a bounded deadline, and distinguishes transient connection loss from authentication/TLS failure. The runner can swap providers without changing browser login logic.

`automation/resolver.py`

`strategies()` creates Playwright locators in the prescribed order. `resolve()` scans the frame tree for each strategy, records matches/misses, rejects ambiguous visible controls, waits for temporary DOM/frame settling, and returns one control or raises a classified error. `present()` is a short bounded probe used for optional controls like rejection markers. This module is shared by production and offline mode.

`automation/runner.py`

`Runner.__init__()` stores validated configuration and starts a redacted manifest. `stage_run()` wraps an operation in attempt/timing records and safe backoff. `authenticated()` checks private markers, identity, and optional API probes in a separate tab. `login()` runs username/password and code-provider steps, confirms successful second factor, and saves state. `action()` executes navigation, form interaction, waits, or captures. `register_file()` validates files and hashes them. `evidence()` produces sanitized diagnostics. `run()` owns Chromium lifecycle, validates/reloads state, runs the ordered workflow, and always writes the final manifest.

`automation/security.py`

`Redactor` registers secrets and masks them in text, JSON values, and URL details. `private_json()` uses restrictive permissions and atomic replacement for state/manifests. `EvidenceHTML` is a fail-closed HTML parser: it retains a limited set of structural tags, removes all attributes except a normalized input type, masks all visible text, and drops risky tags/content. This is safer than searching only for known secrets because real pages contain unknown identifiers.

`automation/offline.py`

`check_html()` launches Chromium with JavaScript disabled, blocks every network request, loads the saved fixture, validates targets, and invokes the production resolver. It returns the attempts for inspection. The HTML fixture must embed frames with `srcdoc` so it is self-contained.

`automation/api.py`

`api_token()` persists a high-entropy local bearer token. `LocalJobService` validates project-relative paths, stores job state, accepts into a bounded queue, and runs one job at a time. `_run_job()` loads config and calls `Runner`. `make_server()` provides health, UI static files, authenticated job submission/status, and safe HTTP errors. `serve()` starts/stops worker and HTTP service. The job queue is process-local by design.

`automation/ui.html`, `automation/ui.js`, `automation/ui.css`

The HTML defines the input form and accessible result/status section. The JavaScript sends a bearer-authenticated JSON request, polls the job endpoint, and updates text through `textContent` so API values are not interpreted as markup. It keeps no token in persistent browser storage. The CSS is purely presentational. These files are included as package data so the API can serve them when installed.

`automation/demo.py`

`FixturePortal` implements a local HTTP server with synthetic login, TOTP challenge, revocable sessions, nested report form, dependent dropdown, and download/print routes. It can simulate rejected codes and transient errors. `acceptance_demo()` runs first-login, reuse, corrupt state, revoked state, changed inputs, deliberate failure, popup PDF, print PDF, and retry cases. It is a mechanics test, not evidence of a real GitHub run.

Other project files

- `scripts/setup.sh` builds a Unix virtual environment and installs `requirements.lock`.
- `scripts/mutate_html.py` applies class/sibling/wrapper/iframe changes to a saved fixture.
- `scripts/test-live.ps1` runs fresh login, reuse, corrupt-state, and alternate-input tests against GitHub using separate ignored state/output paths.
- `tests/test_acceptance.py` exercises browser login, session validity, provider behavior, dependent form fields, capture paths, resolver alternatives, retry rules, redaction, and evidence.
- `tests/test_api.py` exercises authentication, queue bounds, path restrictions, UI behavior, and fixture execution through the API.
- `portals/*.json` are adapter data, not executable Python. `examples/inputs.github*.json` hold variable business values. `examples/saved-form.html` and `examples/targets.json` drive offline resolver validation.
- `requirements.txt` declares direct runtime packages. `requirements.lock` pins the complete install set. `pyproject.toml` configures the Python package and includes the UI assets.

The test methods make the intended guarantees concrete:

- In `tests/test_acceptance.py`, `test_login_reuse_corruption_and_revoked_http_200_session` covers first login, reuse without visiting login, broken JSON, and valid-looking but revoked state. The wrong-code test checks one permitted retry and terminal second rejection; the auto-submit test covers portals that accept a code as soon as it is filled.

- The iframe/dependent-field test changes category/order/date and exercises download, popup print, and current-page print. The transient/evidence test causes a temporary 503, then intentionally breaks a capture target and checks the bundle for credential and fixture-data leakage. The identity/schema test proves that state for the wrong account is not reused.
- Provider tests use a synthetic mailbox response for fresh-message filtering and validate a known TOTP vector. Resolver tests cover current GitHub tag controls, renamed classes, sibling reordering, wrappers, `srcdoc` iframe, label/nearby/structure/CSS fallbacks, and ambiguity.
- In `tests/test_api.py`, separate tests cover authorized submission/status, traversal rejection, bounded queue behavior, health/token creation, the actual browser UI, and a fixture job that runs through the queue and captures a PDF.

The live PowerShell script runs its cases in order: remove a dedicated state file, assert a full-login PDF, assert the next run reused the session with identical input values, replace the state file with invalid JSON and assert recovery, then run alternate JSON and confirm session reuse plus a captured file. It saves a final summary beneath the unique ignored output directory and does not publish a release.

`automation/__init__.py` and `tests/__init__.py` are Python package markers. `.vscode/launch.json` supplies debugger presets for the local demo, live GitHub preview, and offline resolver; it contains paths/arguments, not secrets. `docs/GITHUB_SETUP.md`, `docs/VSCODE.md`, and `docs/VALIDATION.md` explain account setup, local development, and evidence status. The PDFs are generated from the accompanying Markdown with `docs/pdf.css` and Chromium's print-to-PDF engine.

14. Why this approach, and what changes at one hundred portals?

The design keeps the browser flow data-driven but puts schema validation and safety checks around that data. Separate provider and resolver interfaces let authentication or locator behavior change without rewriting workflow orchestration. One sequential browser process and one API worker are easy to inspect and test; they also avoid cross-account races in this assignment.

At one hundred portals, I would first replace duplicated JSON with versioned portal adapters and contract tests from consented, sanitized DOM snapshots. I would isolate credentials and session state by account and origin, add structured metrics for login/session/locator/capture outcomes, and introduce per-portal rate limits and locks. Only then would I add bounded concurrency, cancellation, durable job storage, and operational recovery. Proxy routing is not a substitute for account isolation, permission, or reliable portal contracts.

15. Submission and review checklist

The tracked repository is the code submission: source, tests, adapters, inputs, scripts, README, validation record, and both PDFs. `examples/output/` contains synthetic fixture artifacts. For complete live evidence, copy the reviewed acceptance summary, four manifests, and fresh/alternate PDFs from ignored `runs/` to `submission-artifacts/github-live/`.

Before handing it over:

1. Run `git status --short` and commit the README and study artifacts.
2. Confirm the GitHub repository points to the final `main` commit.
3. Do not add `.env`, `.state/`, `.venv/`, or `runs/`; review live account artifacts before sharing.